

Assigned: 07th December 2024

Due: 21st December 2024 before (11.59PM)

Marks: 20 marks possible

Submission Procedure:

- **Create / debug / test your assignment in Visual Studio**
- Create a **ZIP** file of your CareerCloud solution and upload it to **BlackBoard**.
 - o Please ensure you upload the entire solution
 - o Name your ZIP file according to the guidelines in the '1 – Sundry' slide deck
 - E.g. NET.22-3-Joe-Smith-N#####.zip
- **DO NOT EMAIL YOUR ASSIGNMENT** – The Humber email server will not deliver the assignment and you will not be notified that it was not delivered.

Description:

In this assignment, you will create the business logic layer. This layer will define business specific rules that the application will have to maintain. The rules will dictate the parameters of allowed data to be saved in the system. These rules are listed in the Rules section below.

Steps:

1. Add a new class library 'Class Library' project to the 'CareerCloud' solution called 'CareerCloud.BusinessLogicLayer'. **The framework selected should be .NET 8.0 (Long Term Support).**
2. Add new class to the 'CareerCloud.BusinessLogicLayer' project called 'ValidationException'. This class will encapsulate an exception and add an additional code value.
 - a. The 'ValidationException' class will inherit from the 'Exception' type.
 - b. The 'ValidationException' class will have a constructor that takes two parameters:
 - i. A 'code' of type int
 - ii. A 'message' of type string
 - c. The constructor created in step b will pass the 'message' string parameter to the base class constructor.
 - d. The 'code' parameter passed to the constructor created in step b will be stored as a public property in the 'ValidationException' class.
3. Create a new class in the 'CareerCloud.BusinessLogicLayer' project called 'BaseLogic'
 - a. The definition of 'BaseLogic' can be found here: <https://dotnetfiddle.net/d40VSB>
4. Create a new class in the 'CareerCloud.BusinessLogicLayer' project called 'SecurityLoginLogic'
 - a. The definition of 'SecurityLoginLogic' can be found here: <https://dotnetfiddle.net/itXkhc>
 - b. 'SecurityLoginLogic' contains validation code and authentication logic
 - i. You will not need to create your own 'SecurityLoginLogic' class for the 'SecurityLoginPoco', using the one supplied is sufficient.

5. For each class in the 'CareerCloud.Pocos' project add a corresponding class in the business logic layer 'CareerCloud.BusinessLogicLayer' created in step 1.
 - a. Each class in the 'CareerCloud.BusinessLogicLayer' project will reflect a class in the 'CareerCloud.Pocos' project.
 - b. Each class in the 'CareerCloud.BusinessLogicLayer' project will be named as the poco class name with 'Logic' as the suffix rather than 'Poco'
 - i. E.g. for the 'ApplicantEducationPoco' there will be a corresponding class in the logic layer called 'ApplicantEducationLogic'
 - c. Each class created in the logic layer will inherit from the 'BaseLogic' class implemented in step 3.
 - i. 'BaseLogic' is a generic class. When applying the 'BaseLogic' class to a logic class you will need to provide the specific type for the generic parameter. For example the 'ApplicantEducationLogic' class will inherit from `BaseLogic<ApplicantEducationPoco>`.
 - ii. Because the 'BaseLogic' contains a constructor that takes the interface 'IDataRepository' as a parameter you will need to create a constructor in the logic class that also takes a parameter of 'IDataRepository' and pass it to the base. E.g. the 'ApplicantEducationLogic' class will have a constructor that looks like this:


```
public
ApplicantEducationLogic(IDataRepository<ApplicantEducationPoco>
repository) : base(repository)
```
6. For each rule in the Rules section (see the table below) override the 'Verify' method and implement the code to ensure that none of the rules have been violated.
 - a. In the Rules section the class that the rule is to be implemented in is specified in the column labelled Class, the property in the Property column, the rule in the Rule column and the code (returned if the rule is violated) in the column labelled Code.
 - b. If a rule is broken the logic class should generate an exception.
 - c. The logic class should **validate all rules prior to returning an exception** if one or more of the rules have been broken.
 - i. Use a 'List<ValidationException>' to collect all rule violations.
 - ii. At the end of the 'Verify' method check to see if any exceptions have been created in the list and if there are throw an 'AggregateException' passing in the list of exceptions in the 'AggregateException' constructor.

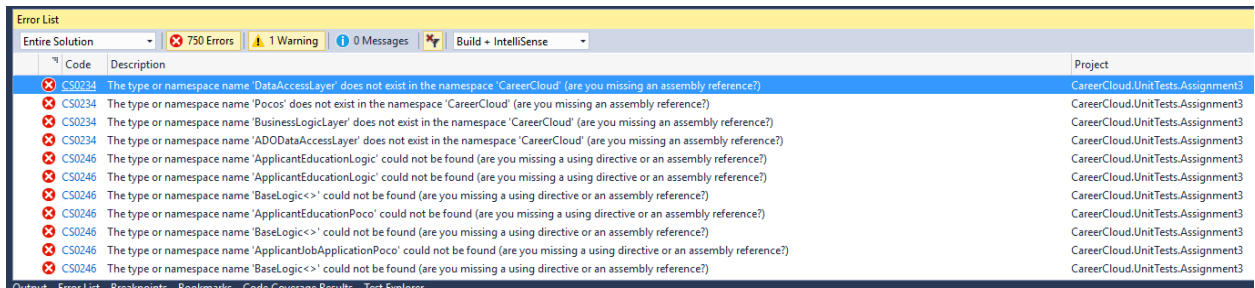
Testing Your Code:

You can test your code before submitting. There is a project on GitHub located here:

<https://github.com/johnhinz/CareerCloudCore.UnitTests.Assignment3.git>

- Clone the solution from GitHub using Visual Studio.
- Open the solution.
- Build the test project

- You should receive an error like the following (these errors are okay):



- Copy the test project folder into your CareerCloud solution folder.
- Add the project to your CareerCloud solution through Visual Studio
 - In your assignments solution explorer window right click and select 'Add' -> 'Existing project'
 - Navigate to the 'CareerCloud.UnitTests.Assignment3.csproj' file you copied previously
- Build the CareerCloud solution.
- Open the Test Explorer window. (Menu: Test -> Windows -> Test Explorer)

You should now be able to run the unit tests on the assignment.

Rules:

Class	Property	Rule	Code
ApplicantEducationLogic	Major	Cannot be empty or less than 3 characters	107
ApplicantEducationLogic	StartDate	Cannot be greater than today	108
ApplicantEducationLogic	CompletionDate	CompletionDate cannot be earlier than StartDate	109
ApplicantJobApplicationLogic	ApplicationDate	ApplicationDate cannot be greater than today	110
ApplicantProfileLogic	CurrentSalary	CurrentSalary cannot be negative	111
ApplicantProfileLogic	CurrentRate	CurrentRate cannot be negative	112
ApplicantResumeLogic	Resume	Resume cannot be empty	113
ApplicantSkillLogic	StartMonth	Cannot be greater than 12	101
ApplicantSkillLogic	EndMonth	Cannot be greater than 12	102
ApplicantSkillLogic	StartYear	Cannot be less then 1900	103

ApplicantSkillLogic	EndYear	Cannot be less then StartYear	104
ApplicantWorkHistoryLogic	CompanyName	Must be greater then 2 characters	105
CompanyDescriptionLogic	CompanyDescription	CompanyDescription must be greater than 2 characters	107
CompanyDescriptionLogic	CompanyName	CompanyName must be greater than 2 characters	106
CompanyJobDescriptionLogic	JobName	JobName cannot be empty	300
CompanyJobDescriptionLogic	JobDescriptions	JobDescriptions cannot be empty	301
CompanyJobEducationLogic	Major	Major must be at least 2 characters	200
CompanyJobEducationLogic	Importance	Importance cannot be less than 0	201
CompanyJobDescriptionLogic	JobName	JobName cannot be null	300
CompanyJobDescriptionLogic	JobDescriptions	JobDescriptions cannot be null	301
CompanyJobSkillLogic	Importance	Importance cannot be less than 0	400
CompanyLocationLogic	CountryCode	CountryCode cannot be empty	500
CompanyLocationLogic	Province	Province cannot be empty	501
CompanyLocationLogic	Street	Street cannot be empty	502
CompanyLocationLogic	City	City cannot be empty	503
CompanyLocationLogic	PostalCode	PostalCode cannot be empty	504
CompanyProfileLogic	CompanyWebsite	Valid websites must end with the following extensions – ".ca", ".com", ".biz"	600
CompanyProfileLogic	ContactPhone	Must correspond to a valid phone number (e.g. 416-555-1234)	601
SecurityLoginLogic	Password	Length must be 10 or greater	700
SecurityLoginLogic	Password	Must contain one of the following characters ("\$", "*", "#", "_", "@")	701
SecurityLoginLogic	PhoneNumber	Cannot be blank	702

SecurityLoginLogic	PhoneNumber	Must conform to a phone number pattern (e.g. 416-555-1234)	703
SecurityLoginLogic	EmailAddress	Cannot be empty and must match a valid email address pattern (e.g. someone@shomewhere.com)	704
SecurityLoginLogic	FullName	Cannot be empty	705
SecurityRoleLogic	Role	Cannot be empty	800
SystemCountryCodeLogic	Code	Cannot be empty	900
SystemCountryCodeLogic	Name	Cannot be empty	901
SystemLanguageCodeLogic	LanguageID	Cannot be empty	1000
SystemLanguageCodeLogic	Name	Cannot be empty	1001
SystemLanguageCodeLogic	NativeName	Cannot be empty	1002

Marking Scheme:

Item	Maximum Mark	Your Mark
Creation of the CareerCloud.BusinessLogicLayer project	1	1
Creation of the ValidationException class	1	1
Implementation of all logic classes	4	4
Required logic classes inherit from base logic and where needed override the Add, Update and Verify methods	4	4
All rules implemented	10	10
Total	20	20